Assignment 1 | Basic Matrix Multiplication in Python, Java and C

Student: [ADD YOUR FULL NAME] Course: Big Data, ULPGC (2026/2027)
Experiment: 16 October 2026 GitHub: [ADD YOUR GITHUB LINK]

Abstract

I compared basic matrix multiplication in Python, Java and C to see how matrix size affects time and memory. I used the same input matrices and the same three-loop algorithm in each language. I tested sizes from 20 × 20 to 200 × 200, with five measured runs for every configuration. At n = 200, the median times were 1133.83 ms (Python), 16.41 ms (Java), and 95.21 ms (C). Java was the fastest in this setup, while C had the lowest observed process memory. These results apply to the programs and settings tested on my laptop.

1. Introduction and objective

I wanted to understand how much slower basic matrix multiplication becomes as the matrices get bigger. My research question was: How does matrix size affect execution time and process memory usage in Python, Java and C? I expected time to increase quickly because the algorithm has cubic complexity. I also expected differences in memory use because each language has a different way of storing and running data.

2. Background

For two square matrices A and B, one output value C[i,j] is found by adding A[i,k] × B[k,j] for every k from 0 to n - 1. The basic algorithm needs three nested loops: over rows, columns and the values being multiplied. As there are n^2 output values and n products per value, the total arithmetic work grows as $O(n^3)$. Dense matrix storage grows as $O(n^2)$ [1]. I used explicit i-j-k loops in all three languages, not library matrix multiplication.

3. Methodology

Inputs and implementation

I tested n = 20, 40, 80, 120, 160 and 200. Matrix entries were integers from 0 to 9, generated with Python random.Random and seed 20261016 + n. Each program read the same input file. Python used lists of lists, Java used long[][], and C used contiguous int64_t arrays. These layouts are different, but the loop order and mathematical operation were the same. The chosen input range avoids integer overflow in this experiment.

Computer and measurement protocol

I used Windows 11, an Intel Core i5-10210U (4 physical cores, 8 logical processors) and 7.72 GiB RAM. The versions were Python 3.12.4, OpenJDK 21 and GCC 16.1.0. The C compiler flags were -std=c11 -O0 -Wall -Wextra. Java ran with the usual JVM JIT compilation enabled. I used two warm-up runs and five measured repetitions per size and language.

The measured time covered one multiplication call, including allocating and zero-initializing the output matrix, but excluding input reading and output writing. The timers were perf_counter_ns in Python, System.nanoTime in Java and QueryPerformanceCounter in C on Windows [2]-[4]. I summarized the five times with the median and interquartile range (IQR).

For memory, an external Python script sampled the whole process RSS about every 2 ms with psutil [5] and recorded the highest observed value. Unlike the time measurement, this includes runtime setup and file I/O. It is one maximum per process, not five repeated memory measurements. The declared limits were 90 seconds and 1024 MiB per worker process.

Correctness

All programs passed a known 2 × 2 multiplication example and tests with identity and zero matrices. For every tested n, I also compared the full output matrix with a separate Python reference. All entries matched exactly.

4. Results

Execution time

Table 1 shows median execution time in milliseconds. The number in brackets is the IQR, also in milliseconds. Each configuration has five timed repetitions.

Matrix n	Python: median (IQR)	Java: median (IQR)	C: median (IQR)
20	1.52 (0.264)	0.18 (0.011)	0.05 (0.012)
40	9.82 (1.074)	0.34 (0.034)	0.39 (0.005)
80	69.10 (7.678)	1.08 (0.004)	3.22 (0.574)
120	230.55 (26.252)	1.76 (0.757)	7.66 (0.203)
160	466.37 (28.358)	8.20 (0.647)	17.47 (0.836)
200	1133.83 (242.159)	16.41 (6.155)	95.21 (2.708)

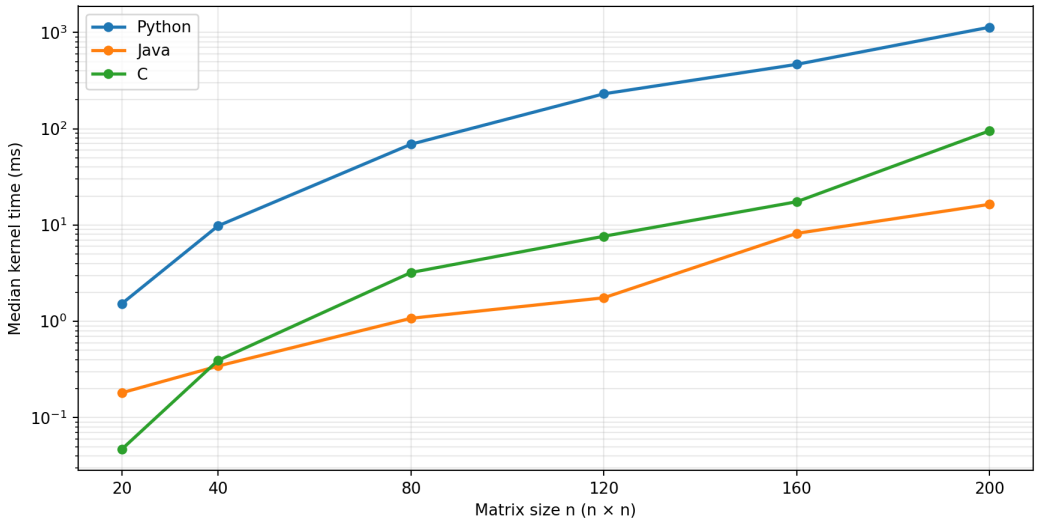


Figure 1. Median multiplication time versus matrix size. The vertical axis is logarithmic; IQR values are in Table 1.

Execution time increased with matrix size in all three programs. For n = 200, Python needed about 1.13 seconds, Java 16.41 ms, and C 95.21 ms. Java was about 69 times faster than Python at this size in these tests. This is a measured comparison, not a claim that Java is always the fastest language.

Process memory

Table 2 shows the maximum observed whole-process RSS in MiB. These values include runtime and I/O overhead.

Matrix n	Python RSS (MiB)	Java RSS (MiB)	C RSS (MiB)
20	12.46	39.08	3.95
40	12.55	40.25	4.22
80	13.18	42.64	4.02
120	14.17	45.33	4.23
160	16.30	47.14	4.40
200	18.44	48.45	4.76

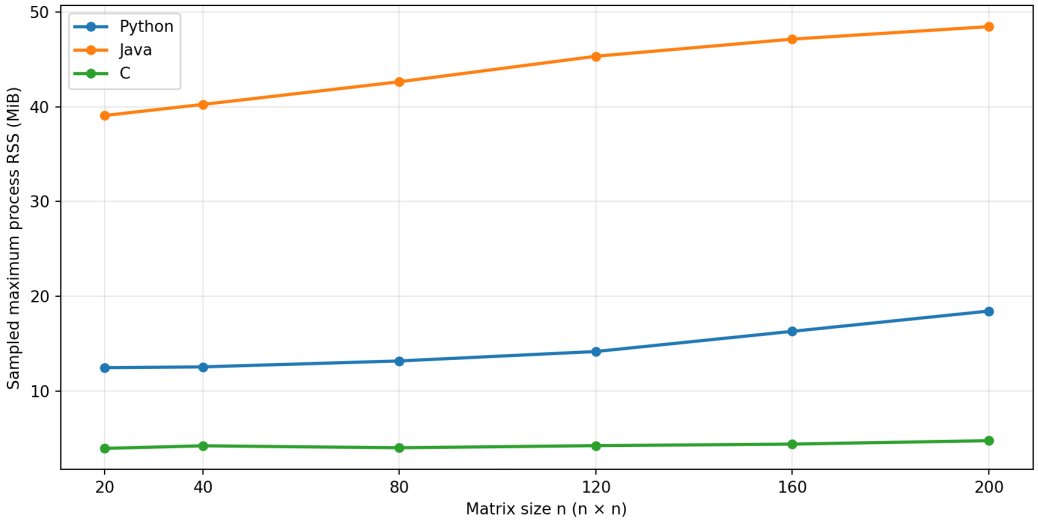


Figure 2. Maximum externally sampled process RSS versus matrix size (MiB).

At n = 200, Python used 18.44 MiB, Java 48.45 MiB and C 4.76 MiB of observed process RSS. C was lowest for every tested size. Java had the largest measurements, which include memory required by the JVM.

5. Discussion and limitations

The results support the expectation that larger matrices take much longer with a basic $O(n^3)$ algorithm. For example, Python increased from 9.82 ms at n = 40 to 69.10 ms at n = 80, roughly seven times slower when n doubled. The theoretical work grows by a factor of eight, but measured times also depend on caching, process scheduling and the runtime.

I expected C to be fast, but Java was faster at the largest matrix sizes. An important reason to be careful is that I compiled C with -O0, so compiler optimizations were disabled, while the JVM could use JIT compilation. Python also has interpreter and object-handling overhead. Therefore, the experiment compares these specific implementations and settings, not the best possible performance of each language.

Memory needs extra care: RSS measures the entire process rather than the size of the matrix arrays. The sampling interval may miss short peaks, and I have only one memory maximum per configuration. Background activity was not fully controlled. I did not set processor affinity or specific JVM garbage collector options. Finally, I only tested one laptop and matrix sizes up to 200.

6. Conclusion

The main conclusion is that execution time rises strongly with matrix size when using straightforward three-loop multiplication. All tests produced correct results, and 200 × 200 was the largest size tested successfully. No timeouts or memory-limit failures were recorded. In this experiment, Java had the fastest median time at n = 200, while C used the least sampled process memory. Different settings, especially C compiler optimization, could change the comparison.

References

[1] ULPGC, Individual Assignments: Matrix Multiplication, Big Data, 2026–2027, pp. 2–4, 8–9.

[2] Python Software Foundation, time — Time access and conversions (Python 3.12), docs.python.org/3.12/library/time.html.

[3] Oracle, System.nanoTime, Java SE 21 API, docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/System.html.

[4] Microsoft Learn, QueryPerformanceCounter function (Windows API documentation).

[5] psutil documentation, Process.memory_info and RSS, psutil.readthedocs.io/en/latest/.

Reproducibility and AI assistance

The GitHub repository includes the source code, README, raw CSV data, summary data, environment information and plotting script. To repeat the study, run python -m pip install -r requirements.txt, then python run_experiments.py.

AI assistance: I used ChatGPT to help plan the experiment, prepare and debug scripts, and edit the English. The measurements shown here came from runs on my computer. I am responsible for understanding the code, checking the results and explaining the project during the defence.